

THE PROJECT

Collateral stays. Credit moves.

Borrow test dUSD on Base Sepolia against dCOL held in a vault on Ethereum Sepolia. The original collateral never crosses the network boundary.

Why it matters

Moving collateral across chains creates extra custody and representation risks. Databaes demonstrates a different flow: keep collateral where it is, authenticate the lock, and issue liquidity on a second network.

What we built

- Solidity vault, lending pool, and authenticated EIP-712 messaging contracts.
- A persistent relay worker with confirmation checks, ordered delivery, and recovery from saved cursors.
- A React loan workspace with wallet discovery, repayment, history, contract evidence, and one-click Sepolia network setup.

Two chains. One verifiable lifecycle.

1. Approve and lock dCOL in the Ethereum Sepolia vault.
2. Relay a signed lock proof after 12 source confirmations.
3. Receive dUSD from the Base Sepolia pool.
4. Repay to release collateral, or default and recover it to the fixed treasury.

[LIVE APP - onehack.shivang.me](https://onehack.shivang.me)

[PUBLIC SOURCE - github.com/shivangtanwar/onehack](https://github.com/shivangtanwar/onehack)

Team Databaes: Shivang Tanwar, Chahat Singh, Aniket Mishra, Punya Mahajan
BML Munjal University

VERIFY THE CLAIMS

Evidence you can inspect.

Recorded public testnet transactions

Lock 100 dCOL

<0x7ea3e7352ca356fa8f715cdf23ed6585c96ed02aa74e1e0f140b239440cb6090>

Issue 50 dUSD

<0x9fa42a452c2d371ae880c9c39ab5a7629445db7c3785ed02ade67de759b5e616>

Recover collateral after default

<0x77d85fa8c65c17490ac2ab5e99f8d47df7af720194cbd01d20c24d762d2221fe>

Exact replay rejected

<0xd5abeb93631fb13cdd29b63e0d2ac3b92f23cac2146206269a61ba3d57ff516c>

Double pledge rejected

<0x07ec6d07d324ec857cfa284bfe11e1ed92cff8abc53db9dc35789184ccd89907>

Expired proof rejected

<0x1c9859b14c8ce1e7de3310a582a025f8e6376321495972db6fe1dbd2983c30ce>

Reproducible verification

21 contract tests pass: lifecycle, repayment/release, default/recovery, replay, double pledge, proof expiry, routing, signer checks, and nonce ordering.

20 browser/wallet tests pass: loan discovery, balance and LTV checks, approval and lock, repayment, history export, mobile layout, network addition, cancellation, and mainnet rejection.

Solidity lint, TypeScript checks, and the production frontend build pass. The shipped frontend and relay runtime dependency audits report zero known vulnerabilities at verification time.

[Inspect all deployment addresses, transaction records, and reproduction commands](#)

Evidence transactions above were recorded on 11 September 2026. Automated checks were rerun on 13 September 2026. Test results are not a security audit.

JUDGE WALKTHROUGH

Try it. Know the boundaries.

A quick walkthrough

- Open the live app without a wallet. Inspect the completed public loan and follow its explorer links.
- Open New loan. Use Add Ethereum Sepolia and Add Base Sepolia to configure your wallet. Each button requests the network and explicitly switches to it.
- For a live loan, obtain test ETH on both networks, connect a wallet, mint test dCOL, choose amounts, then review, approve, and lock.
- Allow several minutes for source confirmations. The demo loan lasts 180 seconds from issuance; repay promptly and wait for collateral return.
- Open Safety & contracts to review recorded attack rejections and the exact deployed addresses.

Honest trust and safety boundaries

Testnet only. The dCOL and dUSD tokens have no monetary value. A single trusted attester signs source events; this is federated messaging, not a trustless bridge. Mock asset valuation assumes equal unit prices and has no price oracle. Contracts are unaudited.

Strict nonce ordering can block a route if its next message expires or cannot be delivered. This version has no safe cancellation protocol for an unrelayed lock. These are documented prototype limitations, not hidden production guarantees.

Public hosting and source

The app is served with HTTPS through Nginx. The relay runs in a non-root container with a read-only filesystem, persistent cursors, restart policy, resource limits, health checks, and a separate secret mount. No private keys are part of the frontend or public repository.

[Security model and limitations](#)

[Operations and recovery guide](#)

[Submission description and demo script](#)